

Entrega 5 — Memoria

Auditoría del Observatorio MinCiencias — USTA 2026-II

Equipo: Kevin, Valentina, Paula

1. Qué hicimos

Partimos de una pregunta simple: ¿cuánto inventa la IA cuando le pedimos cosas sobre nuestro proyecto? La respuesta inicial fue incómoda. De una consulta bibliográfica corrida sin búsqueda ni fuentes, una de cada seis referencias no correspondía a lo que decía ser. De los doce archivos `refs.bib` del proyecto, todos tenían 21 referencias idénticas entre sí y ninguna sección las citaba con `\cite{}`. De las cifras del anteproyecto, dos no tenían respaldo en ningún artefacto del repositorio auditado.

Frente a ese diagnóstico montamos un sistema de fuentes con dos piezas: una memoria semántica (un RAG sobre el corpus documental del proyecto) y una memoria estructural (un grafo de relaciones entre criterios, documentos, glosario, commits y archivos de código). Sobre las dos corren 17 reglas de validación automática que vigilan tres cosas: la consistencia interna del corpus, la coincidencia entre lo documentado y el repositorio auditado real, y la sincronización entre las cifras del propio sistema y los documentos que las citan. El sistema quedó integrado al flujo de trabajo: la validación corre antes de compilar el PDF final, y el RAG se consulta antes de redactar.

2. Las tres lecciones

Lección 1 — El diseño de “recuperar y citar” contiene la invención, pero no la elimina

Cuando evaluamos el sistema con un banco de 20 preguntas con respuesta conocida, el resultado del RAG fue bueno en lo que importa: en 19 de 20 preguntas el sistema no fabricó información. Frente a preguntas cuya respuesta no estaba en su corpus, devolvió fragmentos temáticamente relacionados con scores bajos, sin inventar cifras ni citas. Eso es exactamente lo que el diseño de “recuperar y citar” busca: preferir una respuesta débil pero trazable sobre una inventada con apariencia de verosímil.

La única excepción fue una pregunta sobre si se usa una librería específica en el código del repositorio auditado. El sistema respondió con un fragmento con apariencia de respuesta, aunque el dato no estaba indexado. La causa no era una falla del generador: el corpus son documentos, no código. Ningún ajuste del limpiador podía resolverlo. La corrección es ampliar el corpus.

La lección es matizada: el diseño funciona, pero la cobertura del corpus es un límite real, no un detalle. Si el dato no está, el sistema no debería responder. Y en la mayoría de los casos no responde. En el que respondió mal, la causa fue que el índice no cubría el tipo de contenido que la pregunta pedía. Eso es información útil: saber dónde falla un sistema es más valioso que creer que no falla.

Lección 2 — La peor alucinación no es la que inventa: es la que propaga

Esta fue la lección más incómoda. El corpus del proyecto contenía una cifra errónea con apariencia de verificada: el anteproyecto decía que se detectaron 22 registros con edad mayor a 100 años, cuando el artefacto real del repositorio auditado dice 19. La cifra errónea venía del informe narrativo del repositorio, calculada sobre un universo declarado de 77 237 filas, no sobre el consolidado disponible de 50 891 que el anteproyecto decía estar verificando. Pero al corpus del RAG no le importaba esa distinción: la cifra estaba en un documento indexado, y eso la volvía responsable con cita.

Si alguien le preguntaba al sistema “¿cuántos registros con edad mayor a 100 detectó el repositorio?”, el sistema iba a responder 22 citando el anteproyecto con total seguridad. No era una alucinación en el sentido clásico (el sistema no estaba inventando). Era algo más sutil y más peligroso: el sistema estaba propagando un error del corpus como si tuviera fuente. Y la tenía. Solo que la fuente estaba mal.

La lección es que auditar el sistema no es suficiente. Hay que auditar también el corpus. Un sistema perfecto sobre un corpus con errores produce respuestas malas con apariencia de verificadas, y eso es más difícil de detectar que una alucinación clásica, porque el usuario confía en la cita. La regla R17 que agregamos al final apunta a eso, pero solo parcialmente: vigila que las cifras del propio sistema no queden obsoletas, no que las cifras del anteproyecto sean correctas. Esa segunda parte exige auditoría humana.

Lección 3 — Los defectos importantes suelen estar en dos líneas de código

El banco de evaluación del RAG mostró al principio que 5 de 20 preguntas eran irrespondibles y que varias otras quedaban mal rankeadas. La causa no era TF-IDF ni el diseño del sistema: el indexador tenía un defecto de dos líneas en `rag.py`. La primera borraba las filas de tabla completas antes de indexar (no solo los delimitadores), descartando 329 filas y el 37 % del texto. La segunda eliminaba el carácter `_`, degradando identificadores como `n_atipicos_edad` a `n atipicos edad` antes de indexar.

El efecto medido fue contundente. Antes de la corrección: 145 fragmentos, 105 167 palabras indexadas, 3 de 20 preguntas con la respuesta correcta en el top-1. Después: 150 fragmentos, 116 290 palabras, 9 de 20 en el top-1. Las alucinaciones latentes bajaron de 5 a 1. Una pregunta que antes era irrespondible (la de los 19 registros con edad mayor a 100) ahora está en el índice.

La lección no es “revisar el código”. Es que los defectos importantes no suelen estar donde uno los busca. No estaban en el diseño del RAG, ni en el modelo, ni en la estrategia de validación. Estaban en una línea de limpieza de markdown que nadie había mirado con cuidado porque parecía trivial. Encontrarlos requirió correr el sistema sobre datos reales y dejar que los datos mostraran dónde estaban las fallas.

3. Dónde seguimos fallando

El sistema no está terminado, y es importante decirlo. La regla R15 sigue fallando: la decisión D007 (migrar de TF-IDF a embeddings) fue tomada pero no implementada, y el código sigue usando TF-IDF. Esa falla es deliberada: preferimos dejarla visible y documentada antes que ocultarla con una excepción. La rúbrica pide mostrar un caso donde el sistema falla con su causa técnica, y R15 es exactamente eso.

El corpus todavía tiene errores de contenido. Las cuatro afirmaciones problemáticas del anteproyecto (22 vs 19, 51 % sin año, 24 % vs 26 % en Ingeniería, 36,9 % sin fuente) siguen en el documento. No las corregimos porque el anteproyecto ya está entregado y porque la corrección es un trabajo aparte: el sistema las detectó, las documentó, y ahora puede vigilar que no se repitan, pero la decisión de corregir el documento es de otra fase.

El corpus del RAG todavía no incluye el código del repositorio auditado. Eso significa que preguntas sobre qué librería se usa, qué scripts existen, o cómo está estructurado el código, no son respondibles con el sistema actual. La corrección es ampliar el corpus o migrar a embeddings (D007), que es exactamente la deuda pendiente.

4. Qué haríamos distinto

Tres cosas.

Medir primero, construir después. Nosotros construimos el sistema antes de medir cuánto inventaba la IA. Cuando llegó la rúbrica formal, tuvimos que correr el experimento retroactivamente. No es un error grave, porque el experimento se puede correr en cualquier momento, pero el argumento de “cuánto mejoró el sistema” es más fuerte cuando se tiene una línea base tomada antes de construirlo. Un equipo que empieza sabe que debe medir primero.

Involucrar al corpus en la evaluación desde el principio. Nuestra evaluación inicial del RAG se enfocó en si el sistema acertaba o no, y descubrimos tarde que el corpus mismo tenía errores. Un enfoque mejor hubiera sido, desde el primer día, validar cada afirmación del corpus contra su fuente real antes de meterla al índice. Eso es trabajo manual, pero es la única forma de que el sistema no propague errores.

Escribir el test del caso de fallo antes de arreglar el bug. Cuando encontramos el defecto del indexador, corregirlo fue sencillo (dos líneas). Pero no teníamos un test que dijera “antes de este arreglo, estas 5 preguntas fallan; después, estas 4 siguen fallando y estas otras no”. Construir el test primero hubiera dejado el arreglo mucho mejor documentado. Lo hicimos a posteriori, pero la lección queda.

5. Qué queda para futuros equipos

Este proyecto deja tres cosas a quien lo continúe.

La primera es la arquitectura del sistema de memoria. El RAG, el grafo, las 17 reglas y la integración con `compiler.py` están documentadas en el repositorio, con las decisiones de diseño registradas en `decisiones.yaml` y las fuentes externas en `fuentes_externas.yaml`. Cualquier equipo que abra el repositorio puede entender por qué se eligió RAG local en vez de una API, por qué `networkx` en vez de `Neo4j`, por qué TF-IDF en vez de embeddings, y qué haría falta para migrar. No es una caja negra.

La segunda es el banco de 20 preguntas con respuesta conocida. Está diseñado para que cualquier equipo futuro pueda correr la misma evaluación contra una versión mejorada del sistema y comparar. Si el próximo equipo implementa D007 (embeddings), puede correr el banco de nuevo y ver cuánto mejora el top-1. Eso es la base de un sistema reproducible.

La tercera es la lista de deudas pendientes. D007 (embeddings), la ampliación del corpus para incluir el código auditado, la corrección de las cuatro cifras del anteproyecto, la implementación de un test de regresión para el caso de fallo del indexador. No son mejoras opcionales: son correcciones identificadas por el propio sistema, con causa técnica y evidencia. Cualquier equipo que tome el relevo tiene una lista concreta de dónde empezar.

La lección más grande que nos llevamos es que la actividad “Nada sin fuente” no es sobre herramientas. Es sobre el hábito de preguntar de dónde sale cada afirmación. El sistema que construimos es una forma de hacer esa pregunta de manera sistemática, pero la pregunta es la misma que hace un investigador cuando lee un paper: ¿esto de dónde sale? Y la respuesta tiene que ser una ruta, no una intuición.